

CENTRO UNIVERSITÁRIO ÁLVARES PENTEADO - FECAP

Danilo Oliveira de Almeida - 23011087
Davi Bigotto Lourenço - 23011119
Katiely de Jesus Silva - 23011355
Laura Piveta Pelizzer - 23011129
Matheus Marques Quio - 23011107

Turtle Rescue

Programação Orientada a Objeto

São Paulo
2026

1. Introdução

O projeto **Turtle Rescue** é um jogo educativo desenvolvido em JavaScript com foco na conscientização ambiental e na preservação de tartarugas marinhas. A proposta combina mecânicas de **virtual pet** com **minijogos interativos**, permitindo que o jogador cuide de uma tartaruga, acompanhe sua evolução e aprenda sobre o impacto ambiental no oceano.

O sistema foi desenvolvido de forma modular e orientada a objetos, com o objetivo de demonstrar, na prática, os conceitos fundamentais de **Programação Orientada a Objetos (POO)**, além de garantir organização, escalabilidade e facilidade de manutenção.

2. Arquitetura do Sistema e Implementação em POO

O projeto está estruturado em quatro principais módulos:

- **entities/** → Representa as entidades do jogo (tartarugas e comportamentos)
- **systems/** → Contém os sistemas centrais (lógica do jogo, evolução, missões, ambiente)
- **minigames/** → Implementação dos minijogos
- **collections/** → Gerenciamento de progresso (álbum e figurinhas)

Essa divisão reflete diretamente a separação de responsabilidades no código.

2.1 Classes e Objetos (Implementação Real)

Toda a aplicação é baseada em classes. A classe principal do jogo é:

- **Game (src/systems/Game.js)** - responsável por inicializar e orquestrar o jogo

Durante a execução, o Game instância diversos objetos importantes, como:

- SistemaEvolucao
- SistemaEducacao
- OceanManager
- GerenciadorMissoes
- Album
- Instâncias de tartarugas (TartarugaDeCouro, TartarugaOliva, etc.)
- Minigames (FlappyTurtle, RunnerGame, etc.)

Ou seja, o jogo funciona a partir da **composição de objetos especializados**, cada um responsável por uma parte da lógica.

2.2 Herança (Aplicação no Projeto)

A herança é utilizada em dois pontos principais:

a) Hierarquia de Tartarugas

Classe base:

- Tartaruga

Subclasses:

- TartarugaDeCouro
- TartarugaOliva
- TartarugaDePente

A classe Tartaruga define atributos comuns como:

- saude
- fome
- energia
- experiencia

As subclasses herdam esses atributos e implementam comportamentos específicos, como diferenças em interação e características da espécie.

b) Hierarquia de Minigames

Classe base:

- Minigame (src/minigames/Minigame.js)

Subclasses:

- FlappyTurtle
- RunnerGame
- BubbleJump
- MemoryGame
- Game2048
- entre outros

A classe Minigame define o ciclo padrão:

- iniciar()
- loop()
- atualizar()
- finalizar()

Cada minigame implementa sua própria lógica sobrescrevendo esses métodos.

2.3 Polimorfismo (No Código)

O polimorfismo ocorre principalmente em dois pontos:

a) Método **interagir()**

- Definido na classe Tartaruga
- Sobrescrito nas subclasses

Exemplo de fluxo real:

- O Game chama um método como fazerCarinho()
- Esse método chama interagir()
- O comportamento varia conforme a instância (ex: TartarugaDeCouro ou TartarugaOliva)

b) Minigames

O Game mantém uma referência genérica:

```
this.minigameAtual.iniciar();
```

Independentemente do tipo do minigame, o método chamado é o mesmo, mas o comportamento muda internamente. Isso caracteriza polimorfismo por subtipo.

2.4 Encapsulamento

O encapsulamento é aplicado protegendo a lógica interna das classes:

- Tartaruga controla seus atributos internamente
- Métodos como:
 - alimentar()

-
- ganharExperiencia()
 - atualizarStatus()

impedem alterações diretas nos atributos

Outros exemplos:

- SistemaEvolucao controla regras de evolução
- GerenciadorMissoes controla progressão
- Album controla desbloqueio de figurinhas

Isso garante consistência e evita erros no estado do jogo.

3. Organização e Boas Práticas

3.1 Padronização

- **PascalCase** → Classes (TartarugaDePente, SistemaEvolucao)
- **camelCase** → Métodos e variáveis (ganharExperiencia, atualizarUIInfo)

3.2 Documentação

O projeto utiliza **JSDoc** para documentar:

- Parâmetros
- Retornos
- Responsabilidade dos métodos

Isso facilita manutenção e entendimento do código.

3.3 Modularização

A separação em pastas (entities, systems, minigames, collections) reduz o acoplamento e melhora a escalabilidade.

4. Uso de IA no Desenvolvimento

O projeto utilizou IA como apoio no desenvolvimento, funcionando como **pair programming**.

Ferramentas Utilizadas

- 1. **Antigravity (Google DeepMind):** Utilizada para a arquitetura estrutural (core) e lógica complexa de estados.
- 2. **ChatGPT-4o:** Utilizada para geração de textos educativos, curiosidades e refatoração de métodos auxiliares.
- 3. **Claude:** Utilizada para melhorar prompts, após a tentativa falha em gerar código.

Milestones de Desenvolvimento (Prompts)

Foram utilizadas 10 "Issues" principais para guiar a construção. Abaixo, destacamos as 3 mais críticas:

Fase	Objetivo do Prompt	Resultado Obtido
Arquitetura	Definir GameStateManager e pilha de overlays para navegação fluida.	Sistema robusto que permite abrir o Álbum sem resetar o Home.
Entidade	Criar classe Tartaruga com ciclo de vida, status (fome/saúde) e evolução.	Atributos integrados que mudam dinamicamente com o tempo.
Minigames	Implementar classe abstrata Minigame para garantir polimorfismo.	Possibilidade de adicionar novos jogos (como FlappyTurtle) sem alterar o motor principal.

Decisão Técnica Importante

Na lógica de colisão:

- Uma abordagem usava **classes (Item - Comida/Lixo)**
- Outra usava **funções simples**

A equipe escolheu a primeira por:

- Melhor aderência à POO
- Maior escalabilidade
- Melhor organização do código

5. DevLog (Problemas e Soluções)

Bug 1 – Overlays

Problema: Ao abrir o álbum, a tela principal sumia

Causa: Limpeza indevida do container:

```
this.container.innerHTML = "";
```

Solução: Implementação de overlayStack

```
if (!isOverlay) this.container.innerHTML = "";
```

Bug 2 – Evolução Rápida

Problema: Evolução em poucos segundos

Causa: XP mal calibrado

Solução: Ajuste no SistemaEvolucao

6. Conclusão

O projeto demonstra de forma prática a aplicação de POO em um sistema real. A arquitetura modular permitiu integrar:

- Sistema educativo
- Minigames
- Evolução
- Álbum de progresso

Tudo centralizado na entidade principal Tartaruga.

Principais destaques:

- **Herança:** Minigame → FlappyTurtle
- **Polimorfismo:** método interagir()
- **Encapsulamento:** controle interno dos atributos

O resultado é um sistema organizado, escalável e alinhado com boas práticas de desenvolvimento.